

This project uses **Terraform** and **Jenkins** to automate AWS infrastructure deployment.

1. A developer pushes code to **GitHub**, which triggers a Jenkins pipeline.
2. Jenkins performs code checkout, initializes Terraform (using S3 + DynamoDB backend), selects the environment workspace (dev/uat/prod), and validates the configuration.
3. Based on the selected action (plan/apply/destroy), Terraform provisions infrastructure in sequence:
 - Creates **VPC** and subnets
 - Configures Internet Gateway and route tables
 - Sets security groups for access control
 - Creates **EC2** instances for web servers
 - Provisions **ALB** and registers instances
 - Stores assets in **S3**
4. Once deployed, user requests are routed:
 - User → DNS → ALB
 - ALB performs health checks and forwards traffic to healthy EC2 instances
 - EC2 serves the application, fetching static files from S3 if needed
 - Response flows back through ALB to the user
5. The setup ensures:
 - High availability (multi-AZ deployment)
 - Security (private subnets, restricted access)
 - Automation and repeatability (IaC + CI/CD)

What happens when a user hits the Load Balancer URL?

When someone opens your app (e.g., `http://your-alb-dns`), the request flows like this:

Step-by-step flow

1. User → DNS Resolution

- The browser queries DNS and gets the public IP of your **AWS Application Load Balancer**.

2. Request reaches ALB (Public Subnet)

- The ALB is internet-facing and sits inside your **Amazon VPC** public subnets.
- It listens on:
 - Port 80 (HTTP)
 - Port 443 (HTTPS)

3. Security Group check (ALB level)

- Before anything else, AWS evaluates the **ALB Security Group**:
 - ☐ Allows: `0.0.0.0/0 → 80, 443`
 - ☐ Blocks everything else
- If not allowed → request is dropped immediately.

4. ALB Listener + Rules

- ALB receives the request and evaluates listener rules.
- It forwards traffic to a **Target Group** (your EC2 instances).

5. Health Check validation

- ALB only forwards traffic to **healthy instances**.
- If an instance fails `/health`, it is removed automatically.

6. Forward to EC2 (Private/Public Subnet depending on setup)

- Request is sent to one of your **Amazon EC2** instances.

7. Security Group check (EC2 level)

- EC2 Security Group allows traffic **only from ALB SG** (best practice).
- So:
 - ☐ ALB → EC2 → Allowed
 - ☐ Direct Internet → EC2 → Blocked

8. Application processes request

- Your app decides:
 - Static content → fetch from **Amazon S3**
 - Dynamic content → generate response

9. Response goes back

EC2 → ALB → Internet → User browser

2. How Security Works (Layer by Layer)

Your architecture actually uses **multiple security layers** (this is very good design 🙌)

Layer 1: Network Isolation (VPC)

- Everything runs inside a **Amazon VPC**
- You control:
 - IP ranges
 - Subnets
 - Routing

☞ This ensures **your infrastructure is isolated from others**

Layer 2: Subnet Separation

Public Subnet

- ALB lives here
- Has route to Internet Gateway

Private Subnet (best practice)

- EC2 should ideally live here (your diagram shows mixed, but private is safer)
- No direct internet access

☞ Only ALB is exposed to the internet—not your servers

Layer 3: Security Groups (MOST IMPORTANT)

Think of Security Groups as **stateful firewalls**

ALB Security Group

Inbound:
80, 443 → 0.0.0.0/0

Outbound:
All (to EC2)

EC2 Security Group (ideal setup)

Inbound:
80 → ONLY from ALB Security Group
22 → Only from your IP (or disabled in prod)

Outbound:
All

☞ This ensures:

- Users CANNOT hit EC2 directly

- Only ALB can talk to EC2

Layer 4: Routing Control

- Public Route Table:

`0.0.0.0/0 → Internet Gateway`

- Private Route Table:

`No internet route`

☞ Prevents unintended exposure

Layer 5: S3 Security

Your **Amazon S3** bucket:

- Private (no public access)
- Encrypted
- Access controlled via IAM or VPC endpoints

☞ Users never access S3 directly (unless you allow it)

Layer 6: Health Checks = Availability + Security

- ALB continuously checks `/health`
- If instance:
 - Crashes
 - Becomes slow
 - Returns error

☞ It is removed automatically

End-to-End Flow

```
User
↓
DNS → ALB
↓ (SG check)
ALB Listener
↓
Target Group (only healthy EC2)
↓ (SG check)
EC2 Application
↓
(Optional S3)
↓
Response → ALB → User
```

Important Improvement (Based on Your Design)

Right now your diagram shows EC2 in **public subnets**.

☞ That works, but it's NOT ideal for production.

Better approach:

- Move EC2 → Private Subnets
 - Keep ALB → Public Subnets
 - (Optional) Add NAT Gateway for outbound internet
-

Key Takeaways

- ALB is the **only public entry point**
- Security Groups enforce **who can talk to whom**
- EC2 is protected behind ALB
- Health checks ensure reliability
- S3 is private and secure
- Terraform + Jenkins ensures everything is reproducible